

1. Persiapan dan Instalasi Library

```
1 # Install library yang dibutuhkan
2 !pip install tensorflow pandas numpy scikit-learn sentence-transformers -q
3
4 import pandas as pd
5 import numpy as np
6 import tensorflow as tf
7 from sklearn.model_selection import train_test_split
8 from sklearn.metrics import mean_squared_error, mean_absolute_error
9 from tensorflow.keras.layers import Input, Embedding, Flatten, Concatenate, Dense, Dropout
10 from tensorflow.keras.models import Model
11 from tensorflow.keras.optimizers import Adam
12 from sentence_transformers import SentenceTransformer
13
14 # Mount Google Drive untuk mengakses dataset
15 from google.colab import drive
16 drive.mount('/content/drive')
```

Mounted at /content/drive

2. Memuat dan Memproses Data (Data Preprocessing)

```
1 # Tentukan path dataset
2 movies_path = '/content/drive/MyDrive/dataset sistem rekomendasi/movies.csv'
3 ratings_path = '/content/drive/MyDrive/dataset sistem rekomendasi/ratings.csv'
4
5 # Memuat dataset
6 print("Memuat dataset dari Google Drive...")
7 movies = pd.read_csv(movies_path)
8 ratings = pd.read_csv(ratings_path)
9
10 # Gabungkan data untuk mendapatkan metadata teks pada setiap rating
11 df = ratings.merge(movies, on='movieId')
12
13 # 1. Encoding User ID dan Item ID ke rentang kontinu
14 user_ids = df['userId'].astype('category').cat.codes.values
15 item_ids = df['movieId'].astype('category').cat.codes.values
16
17 df['user_encoded'] = user_ids
18 df['item_encoded'] = item_ids
19
20 num_users = df['user_encoded'].nunique()
21 num_items = df['item_encoded'].nunique()
22
23 print(f"Jumlah User: {num_users}, Jumlah Item: {num_items}")
24
25 # 2. Siapkan label (misal: memprediksi probabilitas interaksi positif / rating >= 3.5)
26 df['interaction'] = (df['rating'] >= 3.5).astype(int)
```

Memuat dataset dari Google Drive...
Jumlah User: 1819, Jumlah Item: 14018

3. Ekstraksi Fitur Teks dengan BERT (Feature Engineering)

```
1 # Inisialisasi pre-trained model BERT yang ringan dan cepat
2 print("Memuat model BERT (all-MiniLM-L6-v2)...")
3 bert_model = SentenceTransformer('all-MiniLM-L6-v2')
4
5 # Membuat fitur teks dengan menggabungkan Judul dan Genre
6 # Memastikan penanganan string kosong atau null
7 movies['title'] = movies['title'].fillna('')
8 movies['genres'] = movies['genres'].fillna('').str.replace('|', ' ')
9 movies['text_feature'] = movies['title'] + " " + movies['genres']
10
11 # Mengekstrak embedding untuk setiap film unik
12 print("Mengekstrak embedding teks...")
13 item_text_embeddings = bert_model.encode(movies['text_feature'].tolist(), show_progress_bar=True)
14
15 # Buat dictionary/mapping dari item_encoded ke vektor BERT
16 item_id_to_encoded = dict(zip(movies['movieId'], movies['movieId'].astype('category').cat.codes))
17
18 # Buat matriks embedding teks yang sejajar dengan indeks item_encoded
19 embedding_dim_bert = item_text_embeddings.shape[1] # Umumnya 384 dimensi untuk MiniLM
20 bert_embedding_matrix = np.zeros((num_items, embedding_dim_bert))
21
22 for idx, row in movies.iterrows():
```

```

23     encoded_id = item_id_to_encoded.get(row['movieid'])
24     if pd.notna(encoded_id) and encoded_id < num_items:
25         bert_embedding_matrix[int(encoded_id)] = item_text_embeddings[idx]
26
27 print(f"Bentuk Matriks BERT: {bert_embedding_matrix.shape}")

```

Memuat model BERT (all-MiniLM-L6-v2)...

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
 WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to

```

modules.json: 100%                               349/349 [00:00<00:00, 22.9kB/s]

config_sentence_transformers.json: 100%           116/116 [00:00<00:00, 9.84kB/s]

README.md: 100%                                  10.5k/10.5k [00:00<00:00, 731kB/s]

sentence_bert_config.json: 100%                   53.0/53.0 [00:00<00:00, 3.76kB/s]

config.json: 100%                                612/612 [00:00<00:00, 55.1kB/s]

model.safetensors: reconstructing file: 100%      90.9MB / 90.9MB, 8.47MB/s

model.safetensors: downloading bytes:            85.0MB, 7.56MB/s

Loading weights: 100%                            103/103 [00:00<00:00, 2440.50it/s]

tokenizer_config.json: 100%                       350/350 [00:00<00:00, 25.9kB/s]

vocab.txt: 100%                                  232k/232k [00:00<00:00, 12.4MB/s]

tokenizer.json: 100%                             466k/466k [00:00<00:00, 18.7MB/s]

special_tokens_map.json: 100%                    112/112 [00:00<00:00, 8.78kB/s]

config.json: 100%                                190/190 [00:00<00:00, 17.1kB/s]

Mengekstrak embedding teks...

Batches: 100%                                    1951/1951 [08:37<00:00, 5.28it/s]

Bentuk Matriks BERT: (14018, 384)

```

4. Membangun Arsitektur Model BERT + NCF

```

1 # Hyperparameters
2 embedding_size = 50
3 hidden_units = [128, 64, 32]
4 dropout_rate = 0.2
5
6 # --- INPUT LAYERS ---
7 user_input = Input(shape=(1,), name='user_input')
8 item_input = Input(shape=(1,), name='item_input')
9 bert_input = Input(shape=(embedding_dim_bert,), name='bert_input')
10
11 # --- EMBEDDING LAYERS ---
12 # Collaborative Filtering Embeddings
13 user_embedding = Embedding(input_dim=num_users, output_dim=embedding_size, name='user_embedding')(user_input)
14 item_embedding = Embedding(input_dim=num_items, output_dim=embedding_size, name='item_embedding')(item_input)
15
16 # Flatten Embeddings
17 user_flatten = Flatten()(user_embedding)
18 item_flatten = Flatten()(item_embedding)
19
20 # Pemrosesan fitur BERT (Mereduksi dimensi teks agar lebih ringkas)
21 bert_dense = Dense(64, activation='relu', name='bert_dense')(bert_input)
22
23 # --- CONCATENATION ---
24 # Menggabungkan fitur CF dan fitur Teks
25 concat = Concatenate()([user_flatten, item_flatten, bert_dense])
26
27 # --- MULTI-LAYER PERCEPTRON (MLP) ---
28 mlp = Dense(hidden_units[0], activation='relu')(concat)
29 mlp = Dropout(dropout_rate)(mlp)
30 mlp = Dense(hidden_units[1], activation='relu')(mlp)
31 mlp = Dropout(dropout_rate)(mlp)
32 mlp = Dense(hidden_units[2], activation='relu')(mlp)
33
34 # --- OUTPUT LAYER ---
35 output = Dense(1, activation='sigmoid', name='prediction')(mlp)
36
37 # Compile Model
38 model = Model(inputs=[user_input, item_input, bert_input], outputs=output)
39 model.compile(optimizer=Adam(learning_rate=0.001), loss='binary_crossentropy', metrics=['accuracy'])
40
41 model.summary()

```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
user_input (InputLayer)	(None, 1)	0	-
item_input (InputLayer)	(None, 1)	0	-
user_embedding (Embedding)	(None, 1, 50)	90,950	user_input[0][0]
item_embedding (Embedding)	(None, 1, 50)	700,900	item_input[0][0]
bert_input (InputLayer)	(None, 384)	0	-
flatten (Flatten)	(None, 50)	0	user_embedding[0...
flatten_1 (Flatten)	(None, 50)	0	item_embedding[0...
bert_dense (Dense)	(None, 64)	24,640	bert_input[0][0]
concatenate (Concatenate)	(None, 164)	0	flatten[0][0], flatten_1[0][0], bert_dense[0][0]
dense (Dense)	(None, 128)	21,120	concatenate[0][0]
dropout (Dropout)	(None, 128)	0	dense[0][0]
dense_1 (Dense)	(None, 64)	8,256	dropout[0][0]
dropout_1 (Dropout)	(None, 64)	0	dense_1[0][0]
dense_2 (Dense)	(None, 32)	2,080	dropout_1[0][0]
prediction (Dense)	(None, 1)	33	dense_2[0][0]

Total params: 847,979 (3.23 MB)
 Trainable params: 847,979 (3.23 MB)
 Non-trainable params: 0 (0.00 B)

5. Persiapan Data Latih dan Pelatihan Model (Training)

```

1 # Siapkan input arrays untuk training
2 X_users = df['user_encoded'].values
3 X_items = df['item_encoded'].values
4
5 # Petakan vektor BERT ke setiap baris transaksi berdasarkan item_encoded
6 X_bert = np.array([bert_embedding_matrix[i] for i in X_items])
7 y = df['interaction'].values
8
9 # Train-Test Split (80% Train, 20% Test)
10 print("Memisahkan data latih dan data uji...")
11 X_u_train, X_u_test, X_i_train, X_i_test, X_b_train, X_b_test, y_train, y_test = train_test_split(
12     X_users, X_items, X_bert, y, test_size=0.2, random_state=42
13 )
14
15 # Proses Training
16 print("Memulai proses training...")
17 history = model.fit(
18     x=[X_u_train, X_i_train, X_b_train],
19     y=y_train,
20     batch_size=256,
21     epochs=10,
22     validation_split=0.1
23 )

```

Memisahkan data latih dan data uji...

Memulai proses training...

Epoch 1/10

723/723 ————— 15s 14ms/step - accuracy: 0.7116 - loss: 0.5560 - val_accuracy: 0.7363 - val_loss: 0.5261

Epoch 2/10

723/723 ————— 11s 15ms/step - accuracy: 0.7541 - loss: 0.5014 - val_accuracy: 0.7398 - val_loss: 0.5224

Epoch 3/10

723/723 ————— 11s 16ms/step - accuracy: 0.7669 - loss: 0.4791 - val_accuracy: 0.7389 - val_loss: 0.5276

Epoch 4/10

723/723 ————— 9s 12ms/step - accuracy: 0.7777 - loss: 0.4605 - val_accuracy: 0.7371 - val_loss: 0.5396

Epoch 5/10

723/723 ————— 12s 14ms/step - accuracy: 0.7896 - loss: 0.4406 - val_accuracy: 0.7353 - val_loss: 0.5532

Epoch 6/10

723/723 ————— 11s 15ms/step - accuracy: 0.8001 - loss: 0.4198 - val_accuracy: 0.7338 - val_loss: 0.5737

Epoch 7/10

```

723/723 ————— 10s 14ms/step - accuracy: 0.8121 - loss: 0.3985 - val_accuracy: 0.7273 - val_loss: 0.6000
Epoch 8/10
723/723 ————— 10s 14ms/step - accuracy: 0.8234 - loss: 0.3765 - val_accuracy: 0.7294 - val_loss: 0.6397
Epoch 9/10
723/723 ————— 11s 15ms/step - accuracy: 0.8354 - loss: 0.3530 - val_accuracy: 0.7278 - val_loss: 0.6797
Epoch 10/10
723/723 ————— 11s 16ms/step - accuracy: 0.8467 - loss: 0.3317 - val_accuracy: 0.7261 - val_loss: 0.7197

```

6. Evaluasi dan Fungsi Rekomendasi

```

1 # Evaluasi pada data uji (Test Set)
2 print("\nMengevaluasi model pada data uji...")
3 loss, accuracy = model.evaluate([X_u_test, X_i_test, X_b_test], y_test)
4 print(f'Test Loss: {loss:.4f}, Test Accuracy: {accuracy:.4f}')
5
6 # Fungsi untuk memberikan Top-N Rekomendasi
7 def recommend_movies(user_id_encoded, top_n=5):
8     # Validasi apakah user valid
9     if user_id_encoded >= num_users or user_id_encoded < 0:
10         print(f"Error: user_id_encoded harus antara 0 dan {num_users-1}")
11         return
12
13     # Buat array item untuk semua film yang ada
14     all_items = np.arange(num_items)
15     user_array = np.full(num_items, user_id_encoded)
16
17     # Ambil fitur BERT untuk semua item
18     all_bert_features = bert_embedding_matrix
19
20     # Prediksi skor probabilitas
21     predictions = model.predict([user_array, all_items, all_bert_features], batch_size=512, verbose=0).flatten()
22
23     # Ambil indeks item dengan skor tertinggi
24     top_indices = predictions.argsort()[-top_n:][::-1]
25
26     print(f"\nRekomendasi Top-{top_n} untuk User Index {user_id_encoded}:")
27     for idx in top_indices:
28         # Kembalikan ke ID asli dan cari nama filmnya
29         try:
30             original_movie_id = list(item_id_to_encoded.keys())[list(item_id_to_encoded.values()).index(idx)]
31             movie_title = movies[movies['movieId'] == original_movie_id]['title'].values[0]
32             score = predictions[idx]
33             print(f"- {movie_title} (Skor Prediksi: {score:.4f})")
34         except ValueError:
35             continue
36
37 # Uji sistem rekomendasi untuk user index 0
38 recommend_movies(user_id_encoded=15, top_n=5)

```

Mengevaluasi model pada data uji...

```

1606/1606 ————— 4s 2ms/step - accuracy: 0.7213 - loss: 0.7186
Test Loss: 0.7186, Test Accuracy: 0.7213

```

Rekomendasi Top-5 untuk User Index 15:

- Big Night (1996) (Skor Prediksi: 1.0000)
- Bad Boys II (2003) (Skor Prediksi: 1.0000)
- Radio Days (1987) (Skor Prediksi: 1.0000)
- Eye of the Tiger (1986) (Skor Prediksi: 1.0000)
- Enemy at the Gates (2001) (Skor Prediksi: 1.0000)